To boldly suggest an overall plan for C++29

Document Number:	P4162R0
Date:	2026-03-27
Audience:	EWG, LEWG, DG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Twenty-nine implementers said the committee is adding features faster than they can ship them. This plan listens.

- **Priority 1:** The Network Endeavor (P4100R0[1]) - the one major new library for C++29
- **Priority 2:** std::execution maturation - give C++26's most ambitious library room to grow
- **Priority 3:** Everything else waits
- **Process:** Feature freeze in August 2028. The final year is for polish, wording, and defect resolution
- **Profiles:** Carved out from the language freeze. The Direction Group's call to action (P3970R0[21]) applies; EWG polishes the design while implementations materialize

The committee may receive more than one plan for C++29. That is healthy. This plan is grounded on P3962R0[2], on twenty years of missing networking, and on working code.

Revision History

R0: March 2026 (pre-Croydon mailing)

- Initial publication.

---

1. Disclosure

The author maintains the libraries that implement the Network Endeavor and has a stake in the outcome. The committee should calibrate accordingly.

C++20 shipped coroutines in 2020. We built a complete coroutine-native networking stack: timers, sockets, DNS, TLS, HTTP. Two libraries, three independent adopters, production trading infrastructure. We also built I/O on senders, because the committee deserved a direct comparison (P4007R0[3], P4014R0[4], P4088R0[5]).

Getting std::execution (P2300R10[6]) into C++26 was a genuine achievement. The exploration of byte-oriented I/O over coroutines fell to a different team. Both bodies of work now exist.

Most references in this paper are the author's own. The author's team produced the published evidence.

Networking is on this plan because the committee asked for it. P2000R4[7] listed networking as a priority. P0592R2[8], R3[8], and R4[8] listed networking as must-work-on-first for C++23. P0592R5[8] dropped it only because "no composable proposal exists."[8] A composable proposal now exists.

---

2. The train keeps running

C++ ships on time. C++11 shipped late. Every standard since has shipped on schedule. C++29 will ship on schedule. But shipping on time means shipping something that works - not a specification that implementers cannot deliver.

3. The implementers spoke

At Kona 2025, twenty-nine compiler and library implementers wrote P3962R0[2], "Implementation reality of WG21 standardization":

> *Full conformance to recent standards remains difficult in practice, with some implementations still working toward C++20 conformance.*[2]

> *When features accumulate faster than they can be fully implemented, integrated, and adjusted, they stack on top of incomplete or inconsistent foundations.*[2]

They asked the committee to "consider ways of slowing down the addition of features" and to "consider alternating feature-focused and consolidation-focused releases."[2]

Stroustrup, in P1962R0[9]: "We are trying to do too much too fast. We can do much or do less fast. We cannot do both and maintain quality and coherence."[9]

Voutilainen, in P3265R3[10]: "The motivation is to ship something that works, and is tested to work. If we have to slow things down to achieve that, so be it."[10]

The Direction Group, in P2000R4[7]: "It is not our aim to destabilize the language with a demand of constant change."[7]

When features stack on incomplete foundations, the standard becomes a specification that implementers cannot deliver.

4. What this plan is for

A focus plan and a consolidation plan. It responds to P3962R0[2] by limiting new work to one library and devoting the remaining bandwidth to letting C++26's major features mature.

This is not P2000[7]. That paper covers the broad landscape. This paper decides what the committee works on first - and what it does not work on yet.

This plan proposes a feature freeze date.

5. Priority order

Under this plan:

1. **Networking (P4100R0[1]) and std::execution maturation.** The only two items that receive priority scheduling in LEWG and LWG. Networking is the one new library. std::execution gets room to grow: integration papers, follow-on work, field-driven adjustments from the execution authors and the community
2. **In-pipeline library work.** Papers already under active consideration continue to completion. No new papers enter the pipeline until the backlog is clear
3. **New library papers.** LEWG considers new papers only when its backlog is clear, and only for design review. LEWG does not forward new work to LWG unless LWG is similarly idle

Under this plan, no new language features enter C++29.

- EWG reviews and polishes language papers when it runs out of other work. Under this plan, EWG does not forward new language features to CWG
- CWG focuses on defect resolution, wording fixes, and the core issues list
- Implementers gain time to finish C++26 before the next wave

The implementers asked for time. This plan gives it to them.

The pipeline is a queue, not a firehose.

6. The Network Endeavor

C++26 shipped execution, reflection, and contracts. Structured concurrency, compile-time introspection, language-level preconditions. Three features pursued for years, delivered.

C++ has been trying to standardize networking since 2005. P0592R5[8] dropped it from the C++26 plan because "no composable proposal exists."

A composable proposal now exists.

P4100R0[1] describes a thirteen-paper series built on two production libraries (Capy and Corosio) with three independent adopters (Boost.MySQL, Boost.Redis, Boost.Postgres) and a production trading infrastructure company. Published code, not a design sketch.

The foundation: P4003R1[11], the IoAwaitable protocol. Two concepts, one type-erased executor, a thread-local frame allocator cache. Six pages.

The bridge: P4126R0[12], "A Universal Continuation Model." Senders invoke any awaitable without allocating a coroutine frame. One I/O implementation, both coroutines and senders consume it, zero allocation for either path. This is the number one language-adjacent priority for C++29.

Supporting work: P4088R0[5], P4089R0[13], P4092R0[14], P4093R0[15], P4095R0[16], P4096R0[17], P4124R0[18], P4125R0[19].

Coroutines serve byte-oriented I/O. Senders serve compile-time work graphs and heterogeneous dispatch. Both serve real domains. This plan treats them as complementary.

One major new library is the responsible answer to P3962R0[2]. Networking is the one. Twenty years overdue. The substrate is ready.

7. Letting the big features grow

std::execution provides structured concurrency, sender composition, and a customization point model that enables heterogeneous dispatch. A specification of this scope does not reach its full potential in one cycle.

This plan gives std::execution that time. LEWG and LWG gain bandwidth to process integration issues, performance discoveries, and follow-on papers from the execution authors as the feature matures in the field. It is the natural second phase of a large feature's lifecycle.

The same applies to reflection and contracts. Foundational additions generate follow-on work. The committee needs capacity for it.

P4126R0[12] lives at the boundary between execution and networking: zero-allocation awaitable consumption from sender pipelines. As both features grow alongside each other, this bridge becomes the integration point. A coherent async story, not two silos.

The best thing the committee can do for std::execution is give it room to grow.

8. The freeze

At Croydon, papers were being updated mid-meeting. The last meeting before a standard ships is not the place for surprises.

Under this plan, LEWG and EWG stop accepting new papers in August 2028. The cutoff applies to new proposals, not revisions of papers already in progress.

The final year - September 2028 through 2029 - is wording review, defect resolution, integration testing, and polish. LWG and CWG have the floor. Design groups shift to issue processing.

P3962R0[2] asked for "alternating feature-focused and consolidation-focused releases."[2] A freeze within the existing three-year cycle achieves the same goal without changing the cadence.

The last meeting before a standard ships should contain no surprises.

9. "One new library - is that enough?"

P3962R0[2] asked for consolidation. One new library plus room for C++26 features to mature is the consolidation the implementers requested. The alternative ignores twenty-nine implementers who said they cannot keep up.

10. "Where is std::execution integration with networking?"

Domain separation. Coroutines for byte-oriented I/O, senders for compile-time work graphs. SG14 published direction in February 2026 (paper number to be added in R1) advising that networking not be built on P2300R10[6].

This plan reserves time for both features to mature. P4126R0[12] is the bridge. Both models gain from it.

11. "Networking has failed before."

Previous attempts lacked coroutines. Type erasure through `coroutine_handle<>`, stackless suspension, symmetric transfer, frame as state - these resolve the structural problems that stalled prior efforts. Two production implementations, three independent adopters. Working code.

12. "What about safety profiles?"

C++ is under sustained external pressure on memory safety. Government reports, media campaigns, competing languages. The committee was designed without executive authority - it cannot pivot quickly. Profiles are the committee's response to a real-time threat, and that makes them categorically different from ordinary language features.

The Direction Group issued a call to action in P3970R0[21], urging implementers to build the Profiles framework and design initial profiles within it.

Profiles are carved out from the language freeze. Safety hardening within the existing language - removing undefined behavior, adding erroneous behavior, library hardening - is consolidation work and fits in Priority 2 today. Profiles as a language feature advance to CWG once a credible implementation ships. Implementations appear to be underway; the committee is better served by waiting until they materialize. Until then, EWG polishes the design.

Working code before wording.

13. “Study groups?”

They continue. The freeze applies to forwarding, not to design work. SG1, SG7, SG21, SG22, SG23 all continue. Their proposals mature during the consolidation cycle and arrive ready for C++32.

14. “Technical Specifications?”

Yes. The freeze applies to the IS working draft. TSes remain available. This is the escape valve.

15. “My favorite idea X?”

This plan reserves bandwidth for one new library and for C++26’s features to mature. Additional work requires demonstrating that it does not displace that capacity.

16. Conclusion

C++29 can be the release that proves the committee listens. One major new library - networking, twenty years overdue, built on working code. Room for std::execution, reflection, and contracts to grow. A freeze that ensures the final year produces a polished standard. And time - for implementers to catch up, for the ecosystem to absorb C++26, and for the committee to deliver quality over quantity.

Acknowledgements

The author thanks Nina Ranns, Erich Keane, Vlad Serebrennikov, Aaron Ballman, and the twenty-five other co-authors of P3962R0[2] for documenting the implementer perspective that motivates this plan. The author thanks Steve Gerbino, Mungo Gill, Mohammad Nejati, Michael Vandeberg, and Klemens Morgenstern for building the libraries and co-authoring the papers that constitute the Network Endeavor. The author thanks Ruben Perez, Marcelo Zimbres Silva, and the Boost.Postgres team for independently adopting the coroutine-native I/O stack.

References

1. P4100R0 - “The Network Endeavor,” Falco et al. https://wg21.link/p4100r0
2. P3962R0 - “Implementation reality of WG21 standardization,” Ranns et al. https://wg21.link/p3962r0
3. P4007R0 - “Senders and Coroutines,” Falco et al. https://wg21.link/p4007r0
4. P4014R0 - “The Sender Sub-Language,” Falco et al. https://wg21.link/p4014r0
5. P4088R0 - “The Case for Coroutines,” Falco et al. https://wg21.link/p4088r0
6. P2300R10 - “std::execution,” Niebler, Baker, Shoop et al. https://wg21.link/p2300r10
7. P2000R4 - “Direction for ISO C++,” Vandevoorde, Hinnant, Orr, Stroustrup, Wong. https://wg21.link/p2000r4
8. P0592R5 - “To boldly suggest an overall plan for C++26,” Voutilainen. https://wg21.link/p0592r5
9. P1962R0 - “How can you be so certain?” Stroustrup. https://wg21.link/p1962r0
10. P3265R3 - “Ship Contracts in a TS,” Voutilainen. https://wg21.link/p3265r3
11. P4003R1 - “The Narrowest Execution Model for Networking,” Falco et al. https://wg21.link/p4003r1
12. P4126R0 - “A Universal Continuation Model,” Falco, Morgenstern. https://wg21.link/p4126r0
13. P4089R0 - “On the Diversity of Coroutine Task Types,” Falco et al. https://wg21.link/p4089r0
14. P4092R0 - “Consuming Senders from Coroutine-Native Code,” Falco et al. https://wg21.link/p4092r0
15. P4093R0 - “Producing Senders from Coroutine-Native Code,” Falco et al. https://wg21.link/p4093r0
16. P4095R0 - “Retrospective: The Basis Operation and P1525,” Falco et al. https://wg21.link/p4095r0
17. P4096R0 - “Retrospective: Coroutine Executors and P2464R0,” Falco et al. https://wg21.link/p4096r0
18. P4124R0 - “Combinators and Compound Results from I/O,” Falco et al. https://wg21.link/p4124r0
19. P4125R0 - “Field Report: Coroutine-Native I/O at a Derivatives Exchange,” Falco et al. https://wg21.link/p4125r0
20. P2464R0 - “Ruminations on networking and executors,” Voutilainen. https://wg21.link/p2464r0
21. P3970R0 - “Profiles and Safety: a call to action,” Vandevoorde, Garland, McKenney, Orr, Stroustrup, Wong. https://wg21.link/p3970r0